

Hospital Patient Journey Analytics Project Plan

Hospital Patient Journey Analytics

SQL KPI Reporting, ML-Based Long-Wait Prediction and Healthcare Business Storytelling

Item	Details
Project type	Two-person healthcare analytics portfolio project
Team members	Owen and Nimi
Owen focus	Python, exploratory data analysis, machine learning, Streamlit dashboard, data storytelling
Nimi focus	SQL database design, SQL cleaning, KPI queries, reporting views, healthcare BI questions
Team GitHub link	[Insert final GitHub repository link here]
Plan version/date	Version 1.1 - 28 June 2026

Project summary

This project builds an end-to-end hospital patient journey analytics system combining a SQL business intelligence layer, Python analysis, machine learning, an interactive dashboard, and management-focused reports. The project identifies bottlenecks in waiting times, patient flow, department pressure, and long-wait risk, then translates the findings into executive and operational insights for hospital management.

Document Control

Version	Date	Author(s)	Changes
1.0	28 June 2026	Owen and Nimi	Initial project plan created.
1.1	28 June 2026	Owen and Nimi	Updated timeline to 01 July - 12 August 2026, moved planning sessions to Sundays, made milestones weekly, added explicit business questions, and replaced project_report.md with reports/executive_summary.md and reports/business_insights.md.

Contents

- 1. Project Specifications
- 2. Project Timeline
- 3. Data Management Plan
- 4. Pipeline Workflow
- 5. Quality Assurance and Testing Plan
- 6. Team Plan / Resource Plan

1. Project Specifications

1.1 Project Topic

Hospital Patient Journey Analytics: SQL KPI Reporting, ML-Based Long-Wait Prediction and Healthcare Business Storytelling. The project analyses how patients move through a hospital journey from arrival to triage, waiting, treatment, admission or discharge, and possible readmission. The final output is an analytics workflow that combines SQL reporting, Python analysis, machine learning, a Streamlit dashboard, and management-focused reporting.

1.2 Problem Statement

Hospitals often experience operational pressure caused by long waiting times, uneven department demand, delayed admissions, inefficient patient flow, and limited visibility across the full patient journey. Without a clear analytics system, it can be difficult for hospital managers to identify where delays occur, which departments are under pressure, which patient groups are most affected, and which journeys are most likely to result in long waits or extended stays. This project addresses that issue by developing a data-driven patient journey analytics system that monitors operational KPIs, answers clear healthcare business questions, and supports evidence-based recommendations.

1.3 Objectives

- Define explicit healthcare business questions that guide the SQL analysis, dashboard pages, and final recommendations.
- Design a relational SQL database to store patient journey, department, admission, discharge, and readmission information.
- Use SQL to clean the data, create analysis-ready views, and calculate operational KPIs.
- Use Python to perform exploratory data analysis and visualise patient flow patterns.
- Train and evaluate a machine learning model that predicts whether a patient is likely to experience a long wait.
- Build an interactive Streamlit dashboard that presents SQL insights, Python visualisations, model results, and recommendations.
- Create executive and business insight reports so the results are presented from a hospital management perspective.
- Document the project clearly so that both team members have visible, evidence-based contributions on GitHub.

1.4 Goal / Target

Main goal: To build a complete hospital patient journey analytics solution that helps users understand patient flow, identify waiting-time bottlenecks, compare department pressure, estimate long-wait risk, and communicate practical recommendations through a clear SQL-to-Python-to-dashboard pipeline.

1.5 Key Performance Indicators (KPIs)

KPI category	KPI	Definition / success measure
Hospital analytics	Average waiting time	Mean patient waiting time in minutes by department, triage level, or time period.
Hospital analytics	Long-wait rate	Percentage of patients with wait_time_minutes greater than 240 minutes.
Hospital analytics	Admission rate	Percentage of encounters resulting in admission.
Hospital analytics	Discharge rate	Percentage of encounters ending in discharge.
Hospital analytics	Readmission rate	Percentage of patients readmitted within 30 days, if available in the dataset.
Hospital analytics	Department pressure index	Combined measure using patient volume, average wait, and long-wait cases.
Project success	SQL deliverables complete	Database schema, cleaning scripts, KPI queries, and SQL views are committed to GitHub.
Project success	ML model evaluated	At least two models are trained and compared using accuracy, precision, recall, F1-score, and ROC-AUC where applicable.
Project success	Dashboard complete	Dashboard includes overview, patient flow, department pressure, SQL insights, ML prediction, and recommendations pages.
Project success	Documentation complete	README, data dictionary, executive summary, business insights report, setup instructions, and screenshots are complete and easy to follow.

1.6 Business Questions

The business questions below will guide the SQL queries, dashboard design, and final management-focused storytelling. They make the project more healthcare BI-focused rather than only technical.

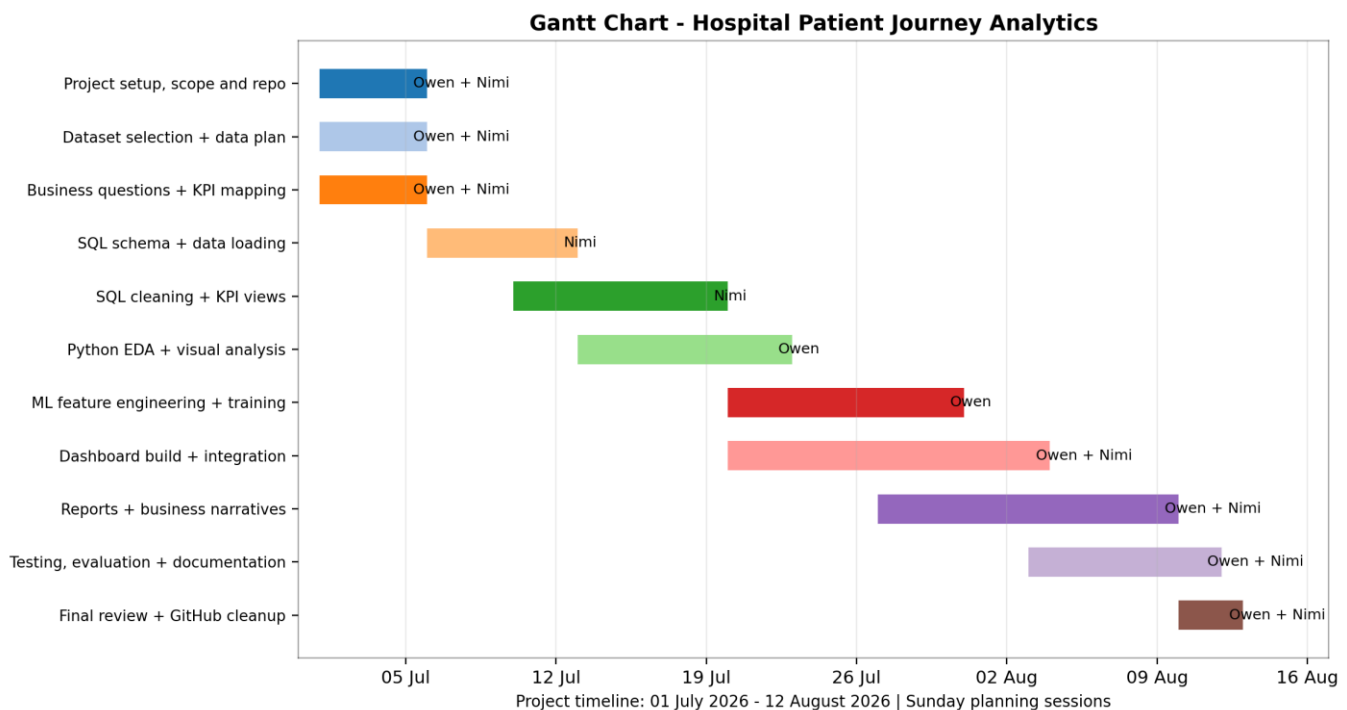
1. Where in the patient journey do the longest delays occur: arrival, triage, waiting, treatment, admission, or discharge?
2. Which departments experience the highest patient pressure when patient volume, average waiting time, and long-wait cases are considered together?
3. What days of the week or times of day show the highest arrival volume and longest waits?
4. What percentage of patients are treated within the target waiting-time threshold, and how does this vary by department or triage level?
5. Which patient groups or triage categories are more likely to experience long waits or longer stays?
6. What factors are most associated with long-wait risk according to the machine learning model?
7. If readmission data is available, which groups or departments show higher readmission patterns?
8. What practical actions could hospital managers take to improve flow, reduce waiting times, and prioritise pressure points?

1.7 Team GitHub Link

Team GitHub Repository: [Insert final GitHub repository link here]

2. Project Timeline

2.1 Gantt Chart



2.2 Project Tasks, Dates, Deliverables, and Assessments

Task	Owner	Start	End	Deliverable / assessment
Project scope, repo setup, and role confirmation	Owen + Nimi	01 Jul 2026	05 Jul 2026	Project specification, GitHub repository, agreed folder structure.
Dataset selection and data management plan	Owen + Nimi	01 Jul 2026	05 Jul 2026	Dataset chosen, metadata table, ethics statement, storage plan.
Business questions and KPI mapping	Owen + Nimi	01 Jul 2026	05 Jul 2026	Business questions mapped to SQL queries, dashboard pages, and final reports.
SQL schema design and data loading	Nimi	06 Jul 2026	12 Jul 2026	SQL create tables script, data import script, raw-to-SQL documentation.
SQL cleaning and KPI views	Nimi	10 Jul 2026	19 Jul 2026	Cleaning queries, KPI queries, views for Python and dashboard.
Python EDA and visual analysis	Owen	13 Jul 2026	22 Jul 2026	EDA notebook, summary plots, initial insights.
ML feature engineering and training	Owen	20 Jul 2026	30 Jul 2026	Model notebook/script, model metrics, feature importance.
Dashboard build and integration	Owen + Nimi	20 Jul 2026	03 Aug 2026	Streamlit app with SQL insight pages, ML output, and recommendations.
Executive summary and business insights writing	Owen + Nimi	27 Jul 2026	09 Aug 2026	reports/executive_summary.md and reports/business_insights.md.
Testing and evaluation	Owen + Nimi	03 Aug 2026	11 Aug 2026	Data quality tests, SQL validation, model evaluation, dashboard user checks.
Final review and GitHub cleanup	Owen + Nimi	10 Aug 2026	12 Aug 2026	Final README, reports, dashboard screenshots, completed repository.

2.3 Weekly Milestones

Milestones are weekly and will be reviewed during the Sunday planning/check-in sessions.

Milestone	Due date	Evidence
Week 1 planning complete	Sunday 05 Jul 2026	Project plan agreed, GitHub repository created, folder structure confirmed, business questions agreed.
Week 2 data and SQL foundation complete	Sunday 12 Jul 2026	Dataset selected, data management notes prepared, SQL schema and data loading started/completed.
Week 3 SQL insight layer complete	Sunday 19 Jul 2026	SQL cleaning queries, KPI queries, and first SQL views completed.
Week 4 analysis and dashboard structure complete	Sunday 26 Jul 2026	Python EDA visuals completed, dashboard skeleton available, SQL views connected or ready for integration.
Week 5 ML and dashboard draft complete	Sunday 02 Aug 2026	Working ML draft, early model evaluation, dashboard pages populated with SQL and Python outputs.
Week 6 reporting and final dashboard draft complete	Sunday 09 Aug 2026	Executive summary draft, business insights draft, dashboard nearly final, documentation updated.
Final submission ready	Wednesday 12 Aug 2026	Clean GitHub repository, final README, final reports, dashboard screenshots, tested app, and contribution statement.

3. Data Management Plan

3.1 Dataset Overview

The project will use a public or synthetic hospital patient journey dataset. The preferred dataset should contain encounter-level information such as patient arrival time, triage level, department, waiting time, admission status, discharge status, treatment duration, length of stay, and readmission flag if available. If a fully patient-level public dataset is not suitable, the team may combine public aggregated waiting-time data with synthetic patient journey records for modelling and dashboard demonstration.

3.2 Data Collection

- Data will be downloaded from a public source or generated synthetically for educational and portfolio use.
- The raw dataset will be stored unchanged in the data/raw folder where legally allowed by the dataset licence.
- Cleaned, analysis-ready datasets will be stored in data/processed.
- SQL scripts will load the raw or processed data into a local database such as SQLite or PostgreSQL.
- All dataset sources, download dates, licence notes, and preprocessing decisions will be recorded in the README and metadata file.

3.3 Metadata / Data Dictionary

Field	Description	Type / example
patient_id	Anonymised patient identifier. No real names or direct identifiers.	Text / P0001
encounter_id	Unique identifier for each hospital visit or encounter.	Text / E0001
arrival_datetime	Date and time of patient arrival.	Datetime
department	Hospital department or service area.	Text / Emergency, General Medicine
triage_level	Urgency classification assigned at triage.	Integer or category
age_group	Grouped age band rather than exact date of birth.	Text / 18-30, 31-45
gender	Patient gender field if included and ethically appropriate.	Text / Male, Female, Other, Unknown
wait_time_minutes	Time from arrival to being seen or treated.	Numeric
treatment_time_minutes	Duration of treatment process.	Numeric
length_of_stay_hours	Total time from arrival/admission to discharge.	Numeric
admission_status	Whether the patient was admitted.	Boolean/category
discharge_status	Outcome at discharge.	Text/category
readmitted_30_days	Whether the patient returned within 30 days, if available.	Boolean
long_wait_flag	Target variable for ML: 1 if wait_time_minutes > 240, else 0.	Binary

3.4 Document Control Using GitHub

Control item	Plan
Repository	[Insert final GitHub repository link here]
Branching system	main for stable work; feature/sql-nimi for SQL work; feature/python-owen for Python, ML, and dashboard work.
Pull requests	Each major change should be reviewed before merging into main.
Commit messages	Use clear messages such as sql: create schema, sql: add KPI views, python: add EDA notebook, dashboard: add ML page, reports: add business insights.
File naming	Use numbered files such as 01_create_tables.sql, 02_cleaning_queries.sql, 01_python_eda.ipynb, and 02_ml_long_wait_prediction.ipynb.
Versioning	Use tags or release notes for final versions, for example v1.0-final.

3.5 README File

The README.md file should explain the project clearly and include:

- Project overview and problem statement.
- Team members and contribution summary.
- Business questions and KPI definitions.
- Dataset source, licence, and data dictionary.
- How to create the SQL database and run the SQL scripts.
- How to run the Python notebooks and Streamlit dashboard.
- Screenshots of the dashboard.
- Model results and limitations.
- Links to reports/executive_summary.md and reports/business_insights.md.

- Final insights and recommendations.

3.6 Security and Storage

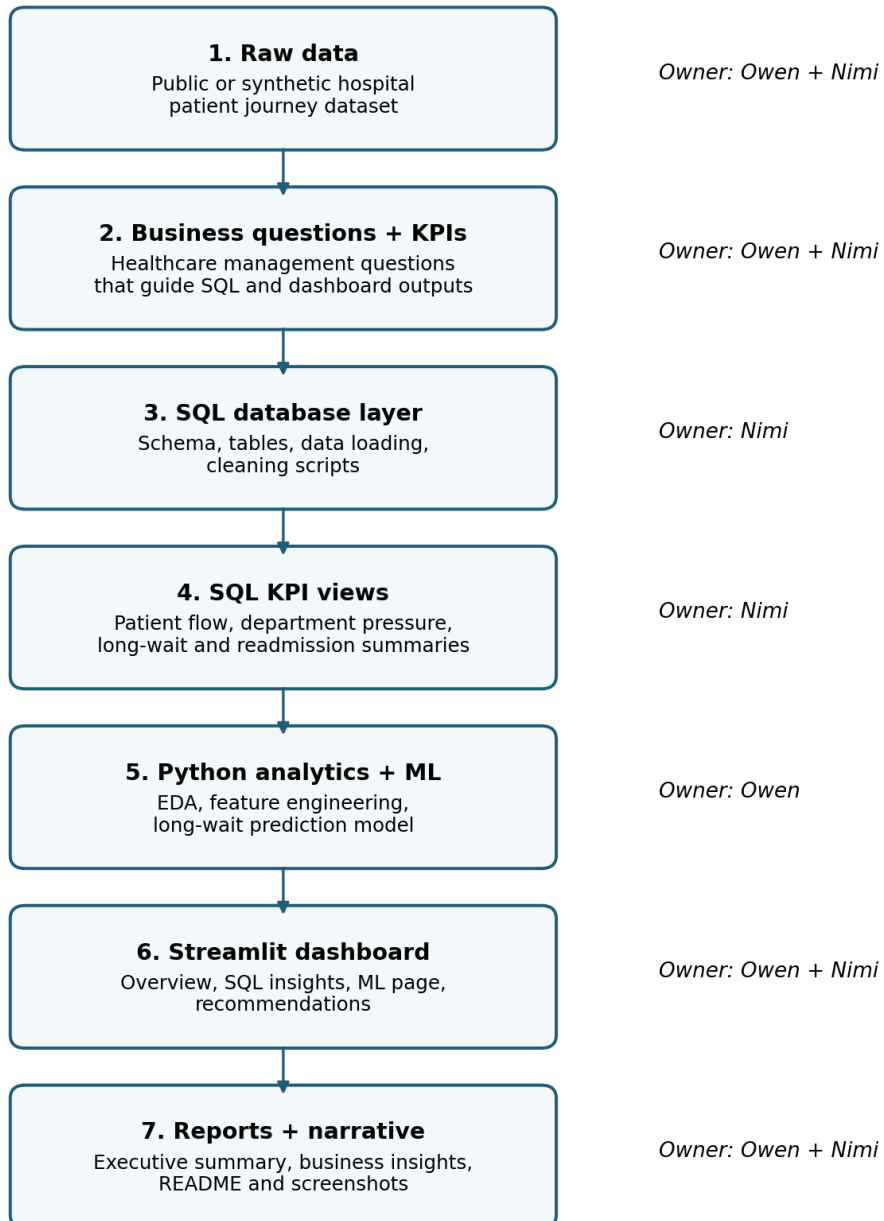
- No direct patient identifiers such as names, addresses, phone numbers, emails, NHS numbers, or exact dates of birth will be stored in the repository.
- Any local database credentials will be stored in a .env file and excluded from GitHub using .gitignore.
- Raw data will only be committed if the dataset licence allows redistribution. Otherwise, the README will provide instructions for downloading it.
- Processed data will be anonymised or synthetic where possible.
- The project will be presented as an educational analytics project, not a clinical decision-making tool.

3.7 Ethical Statement

Ethical question	Project response
1. What data is being used, and is it personal?	The project will use public or synthetic patient journey data. Direct identifiers will not be used. Patient IDs will be anonymised or synthetic.
2. How will privacy and confidentiality be protected?	No real names, contact details, exact addresses, or secret credentials will be included. Sensitive files and credentials will be excluded from GitHub.
3. Could the analysis create bias or unfair outcomes?	The team will avoid using the model as a clinical decision tool. Where demographic fields are available, results will be interpreted carefully to avoid unfair conclusions about patient groups.
4. How will the outputs be used responsibly?	The dashboard and model will be used for learning, portfolio demonstration, and operational insight. Recommendations will be framed as decision-support only, with limitations clearly stated.

4. Pipeline Workflow

Pipeline Workflow: SQL BI Layer + Python ML Layer + Business Storytelling



4.1 Workflow Stages

Stage	Owner	Output
Raw data collection	Owen + Nimi	Dataset stored or referenced with licence notes.
Business question mapping	Owen + Nimi	Business questions linked to SQL KPIs, dashboard pages, and final reports.
Database design	Nimi	Relational schema for patients, encounters, departments, admissions, discharges, and readmissions.
SQL cleaning	Nimi	Cleaned tables, standardised categories, long_wait_flag, and calculated length of stay.
SQL KPI reporting	Nimi	Views such as vw_patient_flow_summary, vw_department_pressure, and vw_ml_patient_features.
Python EDA	Owen	Visual analysis of waiting times, patient flow, departments, and readmission patterns.
Machine learning	Owen	Long-wait prediction model and evaluation metrics.
Dashboard integration	Owen + Nimi	Streamlit dashboard using SQL outputs and Python results.
Reporting and storytelling	Owen + Nimi	README, executive summary, business insights, screenshots, and final recommendations.

4.2 Proposed Repository Structure

```
hospital-patient-journey-analytics/  
|-- data/  
|   |-- raw/  
|   |-- processed/  
|-- sql/  
|   |-- 01_create_tables.sql  
|   |-- 02_load_data.sql  
|   |-- 03_cleaning_queries.sql  
|   |-- 04_kpi_queries.sql  
|   |-- 05_views_for_python.sql  
|-- notebooks/  
|   |-- 01_python_eda.ipynb  
|   |-- 02_ml_long_wait_prediction.ipynb  
|-- src/  
|   |-- data_loader.py  
|   |-- preprocessing.py  
|   |-- train_model.py  
|-- dashboard/  
|   |-- app.py  
|-- outputs/  
|   |-- figures/  
|   |-- model_metrics/  
|   |-- screenshots/  
|-- reports/  
|   |-- executive_summary.md  
|   |-- business_insights.md  
|-- README.md  
|-- requirements.txt
```

4.3 SQL Views for Python and Dashboard

SQL view	Purpose
vw_patient_flow_summary	Summarises patient volume, admissions, discharges, and waiting-time performance.
vw_department_pressure	Ranks departments by volume, average wait, long-wait cases, and pressure index.
vw_business_question_metrics	Connects management questions to KPI outputs used in the dashboard and business insights report.
vw_ml_patient_features	Provides modelling features for Python, including the long_wait_flag target.
vw_readmission_summary	Summarises readmission patterns if the dataset includes readmission fields.

5. Quality Assurance and Testing Plan

Test area	Checks	Owner
Data quality	Check duplicates, missing values, invalid dates, negative waiting times, and inconsistent categories.	Nimi + Owen
SQL validation	Compare SQL KPI outputs with manual sample calculations for selected departments and dates.	Nimi
Business question validation	Check that each business question is answered by at least one SQL query, dashboard chart, or report section.	Owen + Nimi
Python validation	Check dataframe shapes, data types, null values, and consistency between SQL exports and Python inputs.	Owen
ML evaluation	Use train/test split, compare baseline and ML models, report confusion matrix and classification metrics.	Owen
Dashboard testing	Test filters, charts, KPI cards, mobile/desktop layout, and error messages for missing data.	Owen + Nimi
Documentation review	Check README instructions, screenshots, file links, reports, and contribution summary.	Owen + Nimi

5.1 Model Evaluation Criteria

- The model should be compared against a simple baseline, such as predicting the majority class.
- The final model should report accuracy, precision, recall, F1-score, and ROC-AUC where appropriate.
- Recall should be monitored carefully because missing long-wait cases may be more harmful than flagging too many cases.
- Feature importance or model coefficients should be explained in plain language.
- The model should be presented as an analytical prototype, not a clinical or operational automation system.

6. Team Plan / Resource Plan

6.1 Team Roles

Team member	Main role	Responsibilities
Owen	Python, machine learning, dashboard, final analytics story	Python EDA, visualisations, feature engineering, model training, model evaluation, Streamlit dashboard, final insight storytelling.
Nimi	SQL, data modelling, KPI reporting, database documentation, business question alignment	Database schema, SQL data loading, SQL cleaning, KPI queries, SQL views, data dictionary, SQL documentation, SQL-backed business insights.

6.2 Contribution Plan

Work package	Lead	Support	Evidence on GitHub
Project specification and scope	Owen	Nimi	README.md and project planning notes.
Business questions and KPI mapping	Nimi	Owen	README business questions section and sql/04_kpi_queries.sql.
Data management plan and ethics	Nimi	Owen	metadata/data_dictionary.md and README section.
SQL schema and cleaning	Nimi	Owen	sql/01_create_tables.sql and sql/03_cleaning_queries.sql.
SQL KPI queries and views	Nimi	Owen	sql/04_kpi_queries.sql and sql/05_views_for_python.sql.
Python EDA	Owen	Nimi	notebooks/01_python_eda.ipynb.
Machine learning	Owen	Nimi	notebooks/02_ml_long_wait_prediction.ipynb and src/train_model.py.
Dashboard	Owen + Nimi	Owen + Nimi	dashboard/app.py with SQL insights included.
Executive summary and business insights	Owen + Nimi	Owen + Nimi	reports/executive_summary.md and reports/business_insights.md.
Final screenshots and repository cleanup	Owen + Nimi	Owen + Nimi	outputs/screenshots and final README.md.

6.3 Team Meetings / Group Sessions

Meeting type	Frequency	Purpose	Suggested day
Planning session	Weekly	Agree weekly goals, review previous milestone, assign tasks, and discuss blockers.	Sunday
Milestone review	Weekly	Check whether the weekly milestone has been met and update GitHub Issues.	Sunday
Progress check	Weekly	Review commits, check SQL/Python integration, and update issues.	Midweek / flexible
Working session	As needed	Pair work on integration points such as SQL views feeding Python and dashboard outputs.	Flexible
Final review	Once before submission	Test dashboard, check README, verify reports, and confirm deliverables.	Final week

6.4 Team Communication

- Use WhatsApp for quick communication and meeting reminders.
- Use GitHub Issues to record tasks, bugs, and review comments.
- Use pull requests for merging major SQL, Python, dashboard, and report changes.
- Use reports/business_insights.md to collect management-focused insights as the project develops.
- Use reports/executive_summary.md to summarise the final story for a non-technical hospital management audience.
- At the end of each week, each person should commit visible evidence of their work to GitHub before the Sunday review.

6.5 Final Contribution Statement

The final README should clearly state that Nimi built the SQL database, cleaning scripts, KPI queries, reporting views, and SQL-backed business questions, while Owen built the Python EDA, machine learning model, model evaluation, and Streamlit dashboard. Both team members will contribute to the executive summary and business insights report so the project is presented from a hospital management perspective as well as a technical perspective.